

LAN & Address Resolution Protocols

	Protocol	What it does	Layer / Port	Scope	Real-world example (step-by-step)	How an attacker abuses it
	ARP Address Resolution Protocol	Translates an IP address to a MAC address on the local network. Required because Ethernet frames need a MAC destination — IP alone isn't enough to deliver a packet inside a subnet.	L2 (no port)	LAN only (broadcast domain)	Scenario: You ping <code>192.168.1.20</code> from <code>192.168.1.5</code>. Step 1: Your machine has the IP but not the MAC. It broadcasts: <i>"Who has 192.168.1.20? Tell 192.168.1.5"</i> to <code>FF:FF:FF:FF:FF:FF</code>. Step 2: The host with <code>192.168.1.20</code> replies directly: <i>"I'm at BB:BB:BB:BB:BB:BB"</i>. Step 3: Your machine caches the mapping (visible with <code>arp -a</code>) and sends the ICMP echo in a properly-addressed Ethernet frame.	ARP Spoofing / Poisoning: - Attacker sends forged ARP replies to victim ("gateway is at my MAC") and to gateway ("victim is at my MAC") - All traffic now flows through attacker → MITM - Enables: sniffing, SSL stripping, session hijacking, DNS spoofing Tools: <code>arp spoof</code>, <code>ettercap</code>, <code>bettercap</code>
	DNS Domain Name System	Translates a hostname to an IP address via a designated authoritative server. The "proper" name resolution protocol — works locally or across the internet, used for both internal (AD) and external names.	L7 UDP/53 TCP/53	Anywhere routable	Scenario: You type <code>http://intranet.company.local</code> in your browser. Step 1 (DNS): Your machine asks the DNS server: <i>"What's the IP of intranet.company.local?"</i> → reply: <code>192.168.1.20</code>. Step 2 (ARP): To even reach the DNS server (<code>192.168.1.1</code>), your machine ARPs for its MAC. Step 3 (ARP again): Now knowing <code>192.168.1.20</code>, your machine ARPs for its MAC too. Step 4: The HTTP request finally goes out with the correct destination MAC and IP. <i>One simple page load uses DNS once and ARP multiple times.</i>	DNS-based attacks: Zone transfer (AXFR): if misconfigured, dump the entire DNS database DNS cache poisoning: inject fake records (Kaminsky attack) DNS tunneling: covert C2/exfil via TXT records (<code>iodine</code>, <code>dnscat2</code>) Subdomain enumeration: <code>amass</code>, <code>subfinder</code> DNS rebinding: bypass same-origin to attack internal services
	LLMNR Link-Local Multicast Name Resolution	Windows fallback name resolution when DNS returns "no such name". Sends a multicast question to the LAN asking <i>"does anyone know this name?"</i> — and trusts whoever answers first. Enabled by default on Windows.	L7 UDP/5355 TCP/5355	LAN only (multicast)	Scenario: A user types <code>\\fileservr</code> in Explorer (typo — real name is <code>fileserver</code>). Step 1: Windows asks the corporate DNS server: <i>"What's the IP of fileservr?"</i> → DNS replies: <i>"no such name"</i>. Step 2: Windows falls back to LLMNR and multicasts to <code>224.0.0.252</code>: <i>"Anyone here named fileservr?"</i> Step 3: Any host on the LAN can answer. Normally nobody does → request fails. Step 4 (attack): An attacker running <code>Responder</code> replies: <i>"Yes, I'm fileservr, I'm at 192.168.1.66"</i>. Victim connects and sends NTLMv2 hash for authentication.	LLMNR Poisoning: - The #1 most common initial foothold in Windows networks - Tool: <code>Responder -I eth0</code> — listens, answers, captures NTLMv2 hashes - Hashes are cracked offline with hashcat or relayed via <code>ntlmrelayx</code> Mitigation: disable LLMNR via GPO (most orgs forget to)
	NBT-NS NetBIOS Name Service	Even older Windows name resolution, predating LLMNR. Broadcasts a name query on the LAN when both DNS and LLMNR fail. Still enabled by default on every modern Windows version for backwards compatibility.	L5/L7 UDP/137	LAN only (broadcast)	Scenario: Same user types <code>\\printsrv01</code> (an old printer alias that no longer exists in DNS). Step 1: DNS lookup fails. Step 2: LLMNR multicast fails (or LLMNR is disabled). Step 3: Windows falls back to NBT-NS and broadcasts to <code>255.255.255.255:137</code>: <i>"Anyone called PRINTSRV01?"</i> Step 4 (attack): <code>Responder</code> answers as PRINTSRV01. Victim attempts SMB authentication → attacker captures the hash. <i>Same poisoning pattern as LLMNR — different protocol, same fundamental flaw.</i>	NBT-NS Poisoning: - Same tool, same workflow as LLMNR: <code>Responder</code> handles both simultaneously - Often more reliable than LLMNR — many orgs disable LLMNR but forget NBT-NS - Captured hashes used for offline cracking or NTLM relay Mitigation: disable NetBIOS over TCP/IP on every adapter via GPO/DHCP option 1
	DHCP Dynamic Host Configuration Protocol	Assigns an IP address, subnet mask, default gateway, and DNS server to clients when they join the network. The DORA process: Discover → Offer → Request → Ack. Trusts whichever DHCP server responds first.	L7 UDP/67 UDP/68	LAN only (broadcast)	Scenario: You plug your laptop into the corporate network. Step 1 (Discover): Your laptop has no IP. It broadcasts: <i>"Any DHCP servers out there? I need a lease."</i> Step 2 (Offer): The corporate DHCP server replies: <i>"You can have 192.168.1.50, gateway 192.168.1.1, DNS 192.168.1.10, lease 8 hours."</i> Step 3 (Request): Your laptop broadcasts: <i>"I accept the offer from 192.168.1.10."</i> Step 4 (Ack): Server confirms. Your laptop configures its interface and starts using the network. Attack variant: An attacker on the LAN runs a faster/closer DHCP server that hands out: gateway = <i>attacker's IP</i>, DNS = <i>attacker's IP</i>. Every new client unknowingly routes everything through them.	DHCP attacks: Rogue DHCP server: hand out malicious gateway/DNS → instant MITM, no ARP poisoning needed DHCP starvation: exhaust the IP pool with fake MACs (<code>yersinia</code>, <code>dhcpcstarv</code>) so legit clients fail, then offer your own Option 121: push specific routes to victims, redirect target subnets through attacker Mitigation: DHCP snooping on managed switches